arXiv:2604.22266v1 [cs.CL] 24 Apr 2026

Large Language Models Decide Early and Explain Later

Ayan Datta [1] **Zhixue Zhao** [2] **Bhuvanesh Verma** [3] **Radhika Mamidi** [1] **Mounika Marreddy** [3] **Alexander Mehler** [3]

Abstract

Large Language Models often achieve strong performance by generating long intermediate chain-of-thought reasoning. However, it remains unclear when a model's final answer is actually determined during generation. If the answer is already fixed at an intermediate stage, subsequent reasoning tokens may constitute post-decision explanation, increasing inference cost and latency without improving correctness. We study the evolution of predicted answers over reasoning steps using forced answer completion, which elicits the model's intermediate predictions at partial reasoning prefixes. Focusing on Qwen3-4B and averaging results across all datasets considered, we find that predicted answers change in only **32%** of queries. Moreover, once the final answer switch occurs, the model generates an average of **760 additional reasoning tokens per query**, accounting for a substantial fraction of the total reasoning budget. Motivated by these findings, we investigate early stopping strategies that halt generation once the answer has stabilized. We show that simple heuristics, including probe-based stopping, can reduce reasoning token usage by **500 tokens per query** while incurring only a **2% drop in accuracy**. Together, our results indicate that a large portion of chain-of-thought generation is redundant and can be reduced with minimal impact on performance.

1. Introduction

Large language models (LLMs) trained for explicit reasoning have achieved strong performance across a wide range of reasoning tasks, including mathematical problem solving, multiple choice question answering, and tool assisted decision making (Wei et al., 2022; Plaat et al., 2025; Lewkowycz et al., 2022; Kojima et al., 2022), by generating intermediate reasoning steps before producing a final answer. While these traces are typically treated as a monolithic process, the temporal relationship between individual reasoning steps and the model's predicted answer remains poorly understood. Despite their empirical success, the role of these intermediate reasoning tokens in the actual decision-making process of LLMs is unclear. While longer reasoning traces are often associated with improved accuracy, it is not evident whether each generated token faithfully contributes to updating the model's internal belief about the correct answer. In practice, models may continue to generate coherent reasoning even after their beliefs have effectively stabilized. This raises the possibility that a significant portion of reasoning tokens are redundant to the reasoning and do not influence the final output.

Recent studies have questioned the faithfulness of chain-of-thought explanations. Prior work has shown that models can produce correct answers accompanied by incorrect or misleading rationales (Wan et al., 2024; Wiegreffe & Pinter, 2019), and that reasoning traces can be altered or shortened

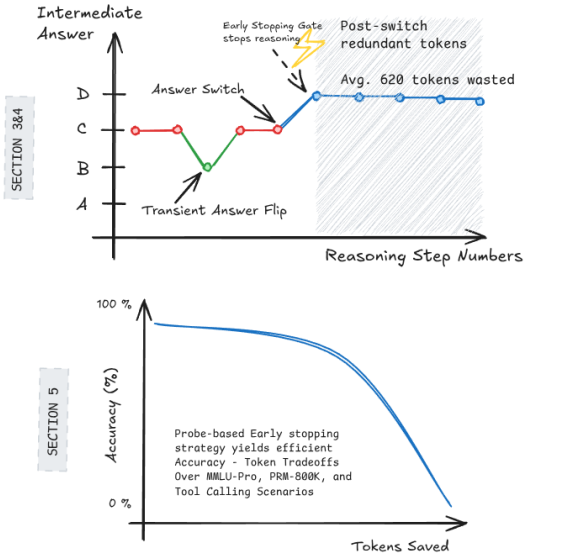


Figure 1. Top: illustration of an obtained answer trajectory for a multiple-choice task. A brief transient flip (answer C → answer B → answer C) reflects local instability, whereas the later sustained transition to D constitutes a genuine answer switch. We study this in Section 4. Bottom: Early-stopping to save token usages with (detailed figures are in Section 5)

[1]IIIT Hyderabad [2]University of Sheffield, UK [3]Goethe University, Frankfurt am Main, Germany. Correspondence to: Ayan Datta <advin4603@gmail.com>.

Preprint. April 27, 2026.

with little impact on model accuracy (Zhou et al., 2023; Wang et al., 2022). Together, these findings suggest that textual reasoning may not reliably reflect the underlying computations driving a model's decisions. However, most existing analyses treat reasoning as a static object, focusing on the presence or absence of chains of thought, rather than examining how a model's answer evolves during the reasoning process. On the other hand, the cost of verbose reasoning has become increasingly important in practice. Long reasoning traces increase inference latency, computational cost, and energy consumption, reduce output conciseness, and can complicate deployment in tool-based or agentic systems (Yao et al., 2022; Karpas et al., 2022). Despite these concerns, there has been limited work investigating how models' predictions stabilize over the course of the reasoning process.

To address this, we use *forced answer completion*, a framework that intervenes on partial reasoning prefixes to elicit the model's predicted answer at intermediate steps. By applying this procedure throughout a reasoning trace, we obtain a *dynamic answer trajectory* that records how the predicted answer changes over reasoning steps generated. This makes it possible to measure answer switches, transient answer flips, and the point at which the predicted answer reaches its final decision.

Using answer trajectories, we empirically characterize the temporal dynamics of predicted answers throughout the model's reasoning process across multiple task settings.[1] We find that predicted answers change in only **32%** of queries on Qwen3-4B, indicating that in the majority of cases the model's answer is fixed from the earliest stages of generation. When answer changes do occur, they typically happen early in the reasoning trace. Despite this early stabilization, the model continues to generate substantial additional reasoning, producing on average **760 tokens after the final answer switch**, and up to **1000 tokens** in some cases. This reveals a systematic temporal gap between when a model's decision is made and when its explanation is produced.

2. Related Work

Faithfulness and Necessity of Chain-of-Thought. Chain-of-thought (CoT) prompting improves performance on many reasoning tasks by encouraging models to generate intermediate steps before giving a final answer (Wei et al., 2022; Kojima et al., 2022). However, recent work questions whether these reasoning steps truly reflect how models reach their final answers. Prior studies show that models can produce correct answers even when their explanations are incorrect, misleading, or added after the fact (Wan et al., 2024). Other work finds that reasoning traces can be edited, rewritten, or significantly shortened with little effect on accuracy, suggesting that not all generated reasoning is necessary for the final prediction (Zhou et al., 2023; Wang et al., 2022). Together, these results suggest that many reasoning tokens may not directly influence the model's decision.

Most existing analyses treat CoT as a static output and focus on whether a final explanation is present, well-formed, or faithful. In contrast, our work studies reasoning as a dynamic process, examining how a model's predicted answer changes as reasoning tokens are generated.

Efficient Reasoning and Token Economy. As reasoning-enabled models become more widely deployed, the computational cost of long reasoning traces has emerged as a practical concern. Verbose reasoning increases inference latency, memory usage, and energy consumption, and can complicate deployment in tool-augmented or agentic systems (Yao et al., 2022; Karpas et al., 2022). Several recent works have therefore explored methods for reducing reasoning cost while preserving performance. These include approaches that compress (), distill (), or restructure reasoning traces (), as well as methods that encourage models to produce shorter or more selective chains of thought (Zhou et al., 2023). More recent studies have begun to explicitly analyze reasoning token wastage, showing that many generated reasoning tokens contribute little to final answer accuracy and may constitute post-decision explanations rather than necessary computation (Bogdan et al., 2025). Related work has also examined adaptive computation and efficiency-oriented decoding strategies that aim to balance reasoning depth against computational cost (Lanham et al., 2023b). However, these approaches typically rely on heuristic stopping criteria or global constraints on reasoning length, without directly measuring when a model's answer has effectively stabilized.

Answer Dynamics and Early Stopping. A smaller line of work has investigated intermediate predictions during generation, often in the context of calibration (Mei et al., 2025; Mao et al., 2026), uncertainty estimation (Mei et al., 2025; Yoon et al., 2025; Zhang & Zhang, 2025; Mao & Venkat, 2026), or self-consistency (Wang et al., 2022; Liang et al., 2024). These methods implicitly acknowledge that a model's belief about the correct answer may evolve, but they do not directly characterize the temporal structure of answer changes within a single reasoning trace. Our work builds on these insights by explicitly tracing model predictions at intermediate reasoning steps.

In summary, prior work has established that CoT reasoning is often unfaithful, unnecessarily verbose, and costly, but has largely analyzed reasoning as a static object or focused on

[1] Unless stated otherwise, all statistics reported in this section are computed using Qwen3-4B and averaged over all datasets considered.

global efficiency constraints. In contrast, our work offers a temporal, answer-centric analysis of reasoning, quantifying answer switches, post-decision token generation, and the potential gains from early stopping.

3. Methodology

3.1. Problem Setup and Notation

We study tasks in which a language model is given an input context and produces a final answer accompanied by an explicit reasoning trace. The input context is denoted by C, which includes the conversation history, task prompt, and any additional information provided to the model. Let R denote the reasoning trace generated by the model. We represent R as a sequence $R = (R_1, R_2, \ldots, R_n)$, where each R_i corresponds to a distinct unit of reasoning. In this work, we assume individual sentences as the basic units of reasoning for analysis purposes, consistent with prior work on natural language rationales and chain-of-though explanations (Wei et al., 2022; Lampinen et al., 2022). Recent reasoning-oriented models explicitly separate reasoning from the final answer. In particular, models inspired by Shao et al. (2024) employ special `<think>` tags to delimit reasoning content. Under this formulation, the final assistant output can be written as: `<think>`R`</think>`A, where A denotes the final answer.

3.2. Forced Answer Completion

Inspired by Lanham et al. (2023a), we analyze the evolution of model predictions over the course of a reasoning trace. Given an input context C and a reasoning trace $R = (R_1, R_2, \ldots, R_n)$, we estimate the intermediate answer the model would predict after each step by explicitly intervening to halt further reasoning and elicit an immediate final answer. Concretely, after generating the partial reasoning trace $(R_1, \ldots, R_i)$, we append the end-of-thinking token `</think>` followed by an answer prefix P, yielding the prompt

$$C\texttt{<think>}R_1R_2 \ldots R_i\texttt{</think>}P.$$

Sampling the model conditioned on this prompt produces an intermediate answer A_i. Repeating this procedure after each reasoning step yields an *answer trajectory* $T = (A_0, A_1, \ldots, A_n)$, where A_0 denotes the answer generated without any reasoning steps and A_i denotes the answer generated after step R_i. The answer prefix for each task is listed in Section B in the Appendix.

3.3. Measuring Decision Changes

Given an Answer Trajectory $T = (A_0, A_1, A_2, \ldots, A_n)$ induced by forced answer completion, we define a set of metrics that characterize the temporal behaviour of the model's predicted answer over the reasoning steps $R = (R_1, R_2, \ldots R_n)$.

3.3.1. Final answer switch index.

We define the index of the last answer switch as the index after which the model's intermediate answer does not change. We calculate this as:

$$t^\star \;=\; \max\big\{\, i \in \{0, \ldots, n-1\} \;\big|\; A_i \neq A_{i+1} \,\big\}, \qquad (1)$$

with the convention that $t^\star = -1$ if no such index exists (i.e., $A_0 = A_1 = \cdots = A_n$).

3.3.2. Answer Switches

We calculate the number of times the model switches its answer from one to another. We do so using the following:

$$\text{AnswerSwitches}(T) = \sum_{i=0}^{n-1} [A_i \neq A_{i+1}] \qquad (2)$$

Where $[\cdot]$ denotes the Iverson bracket, which evaluates to 1 if the condition is true, else 0.

3.3.3. Transient Answer Flips

We also detect cases where the predicted answer briefly switches to an alternative, e.g., for one more step, and then switches back. We refer to these locally inconsistent trajectories *Transient Answer Flips (TAFs)*. That is, a transient answer flip is a short-lived deviation from a dominant answer, lasting a small number of reasoning steps (k), after which the model returns to the original answer. Given an Answer Trajectory $T = (A_0, A_1, A_2, \ldots, A_n)$, we calculate the number of TAFs as follows:

$$\text{TAFs}(T) = \sum_{i=0}^{n-k-1} \begin{bmatrix} \exists\, \ell \leq k \text{ s.t. } A_i = A_{i+\ell+1} \\ \wedge\ \forall j \in \{1, \ldots, \ell\},\ A_i \neq A_{i+j} \end{bmatrix} \qquad (3)$$

Where k is the max length of TAFs we want to detect. We set $k = 3$ as a reasonable default.

3.4. Answer Trajectory Denoising

A high frequency of TAFs indicates that $A_i \neq A_{i+\ell}$ for a small number of further reasoning steps ℓ, despite $A_i = A_{i+\ell+1}$. That is, the model briefly deviates from an answer and then quickly returns to its previous value. Such short-lived reversals are unlikely to correspond to changes in the model's final decision, since they do not persist long enough to affect the eventual output. Instead, they typically reflect local instability in the intermediate reasoning trace, arising from transient token-level fluctuations or sensitivity to nearby context, rather than deliberate revisions of the underlying conclusion.

Motivated by this observation, we aim to filter out these short-lived deviations in the answer trajectory and retain only answer changes that are sustained over multiple consecutive reasoning steps. To this end, we apply a simple causal smoothing procedure that suppresses brief oscillations while preserving persistent answer switches. This denoising removes excess answer switches that do not meaningfully correspond to decision changes, but instead arise from momentary answer flips. As a result, the estimated final answer switch becomes less sensitive to noisy local fluctuations, revealing that even more post-decision reasoning tokens can be saved once such spurious flips are discounted.

Hold-for-k Smoothing Given an answer trajectory $T = (A_0, A_1, A_2, \ldots, A_n)$, we define a causal smoothing operator, referred to as *hold-for-k*. The operator enforces persistence by requiring an answer to remain unchanged for k consecutive reasoning steps before it is accepted as a genuine answer switch. Intuitively, this prevents brief, noisy answer flips from being treated as meaningful changes.

Formally, the smoothed answer trajectory $\tilde{T}^{(k)} = (\tilde{A}_1^{(k)}, \ldots, \tilde{A}_n^{(k)})$ is defined recursively as:

$$\tilde{A}_i^{(k)} = \begin{cases} A_i, & \text{if } A_{i-j} = A_i \ \ \forall j \in \{0, \ldots, k-1\}, \\ \tilde{A}_{i-1}^{(k)}, & \text{otherwise.} \end{cases} \tag{4}$$

This construction is causal, as the smoothed answer at step i depends only on the current and previous reasoning steps and does not rely on future information. This property is essential for online or streaming settings, where answer trajectories are observed incrementally as the model generates its reasoning. In such settings, non-causal smoothing would require access to future steps and therefore cannot be applied in real time. By enforcing causality, hold-for-k smoothing enables on-the-fly monitoring of answer trajectories, allowing sustained answer switches to be detected as they emerge during generation rather than retrospectively.

3.5. Tokens After Final Answer Switch

Given an answer trajectory $T = (A_0, \ldots, A_n)$, we measure the number of tokens generated after $t^\star$. Let T_i denote the cumulative number of generated reasoning tokens up to step i, with $T_0 = 0$ and $T_n = T_{\text{total}}$. We define: $T_{\text{after}} = T_{\text{total}} - T_{t^\star+1}$. This quantity captures the amount of generated tokens that would be saved by halting reasoning immediately after the final answer switch, without altering the model's predicted answer.

3.6. Datasets

We evaluated a total of four tasks. For each task, we report per-example trajectories $(A_0, \ldots, A_n)$ under forced completion and apply identical evaluation and monitoring procedures.

Multiple-choice Question Answering (MCQ) MCQ is commonly used for reasoning evaluation, where each example consists of a question and a small fixed set of candidate options, typically $\{A, B, C, D\}$. We use the datasets from (Wang et al., 2024), sampling the correct answer choice and three other distractor choices. For these datasets, we treat the answer space as the finite set of provided options. At each reasoning step i, we record the option with the highest probability under forced answer completion and use this option as the per-step label A_i.

Numeric-answer We use the dataset from (Lightman et al., 2023), where correct responses are predominantly numeric and typically represented by a single token (e.g., an integer or short numeric constant). For these tasks, we record the highest-probability token under forced completion at each generation step, yielding an answer trajectory $A_0, \ldots, A_n$.

Search-query We evaluate on a closed-book question-answering dataset from (Monteiro et al., 2024) augmented with an explicit search-tool interface. Each example consists of a question paired with a collection of background documents. During generation, the model is prompted to invoke a search tool by producing an appropriate search query as a tool argument.

For analysis, we treat the generated search query itself as the answer. When forced answer completion causes the model to emit a search query, we extract the query text and compare it to the query produced under full generation using the answer label mapping rules. We do not execute the search; instead, we compare the generated queries directly and use query similarity as a proxy for retrieval equivalence.

During forced answer completion, we allow the model to fully generate the search query using the same sampling settings as used for the reasoning trace. To compare two queries obtained at reasoning steps i and j, denoted A_i and A_j, we embed each query using a fixed query encoder to obtain vector representations $\mathbf{a}_i$ and $\mathbf{a}_j$. We consider the two queries to be equivalent if their cosine similarity exceeds a fixed threshold.

$$A_i = A_j \iff \frac{\mathbf{a}_i^\top \mathbf{a}_j}{\|\mathbf{a}_i\| \, \|\mathbf{a}_j\|} \geq \gamma, \tag{5}$$

where $\gamma = 0.9$ in all experiments.

Tool-selection Tool selection is a fundamental challenge in agent-based systems. We evaluate the reasoning capability of LLMs using tasks in which the model is given a fixed set of tools and must select exactly one tool as part of its response (QuotientAI, 2025). At each reasoning step, we extract the tool identifier produced by the model when prompted to emit a tool call. The selected tool is recorded as